Module 4: Classification and Composition

Topics

1. Coupling and Cohesion (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/s3-commentary.html?ou=693191#I)
2. Choosing Composition or Classification (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/s3-commentary.html?ou=693191#II)
3. Aggregation and Acquaintance (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/s3-commentary.html?ou=693191#III)

I. Coupling and Cohesion

One way to evaluate the design of a method, object, or even complete system is to consider its use of coupling and cohesion. Coupling and cohesion affect the structural integrity of a program, and thus impact the overall extensibility, stability, and maintenance of the program. These properties are hard to quantify, though some systems that measure program metrics, such as *Structural Analysis for Java* (http://www.alphaworks.ibm.com/tech/sa4j) , do attempt to quantify them. More important is their usefulness in helping you to rate the quality of two design options. This module will use coupling and cohesion to evaluate overall design options such as when to use classification and composition, and how to use aggregation or acquaintance relationships.

In a larger sense, simply knowing the coupling and cohesion characteristics of a program as it is being built will help a programmer understand the behavior of a program. It will also help programmers find programming problems and make better

decisions as they implement a program. It is with these issues in mind that we will cover coupling and cohesion.

A. Definition of Coupling and Cohesion

Coupling is the degree to which one object or method needs to know, or is given the opportunity to know, something about another object or method. For example, when one object can affect the internal details of another object, those two objects are considered to be tightly coupled. Coupling can occur in many ways in Java. It can come about through the use of a reference or through an extension of an object. The most obvious example of a tightly coupled system is one in which the variables from one object are made public and manipulated by another object. This is shown in the following Person class:

```
public class Person {
String _name;
int _age;

public Person (String name, int age) {
_name = name;
if (age > 0 && age < 120)
_age = age;
else
_age = 0;
}

public void setAge(int age) {
if (age > 0 && age < 120)
_age = age;
else
_age = 0;
}
}

class UseAge {
```

```
public static void main(String[] args) {
Person p = new Person("John Doe", 25);
p._age = -15;
}
}
```

In the above program, the programmer has not made the variable _age private, so any program can access this variable. This _age variable is used in the UseAge class's main() method to unsafely set the _age to an invalid value of -15, which violates a class invariant. UseAge is coupled to the Person class, in that changes to the Person class need to be accounted for in the UseAge class. This is an extreme example of high coupling, but one that does exist in many programs.

Cohesion is the degree to which a class or method has simple, well-defined limits on its purpose and/or functionality. A lack of cohesion is like a lack of focus. The class shown below is a simplified version of one presented at the Microsoft 2005 Academic Discussion to illustrate an instance in which poor programming had made it into a production system. This example highlights a method that lacks cohesion. A single setValue() method is implemented in a Person class. This method uses a flag value to determine what variable to set in this method. Note that even though the _age variable in this method is an int—so that the method will work with any data—the value will be passed in as a String. This system would be much more robust if separate methods were written for each variable in the Person class, and that the set methods for the variables were required to take parameters of the correct class.

```
public class Person {
public enum VariableType {AGE, NAME};

String _name;
int _age;

public void setVariable(VariableType type, String value) {
if (type == VariableType.AGE) {
int iValue = Integer.parseInt(value);
```

```
if (iValue > 0 && iValue < 125)
_age = iValue;
else
_age = 0;
}
else if (type == VariableType.NAME) {
_name = value;
}
}
}
```

It is important to remember that coupling and cohesion are factors that come into play when designing both classes and methods. Although this module will generally deal with problems in classes, it is important to remember that these problems can be present in classes and methods, and that the design of each comes with its own unique problems.

In general, the more loosely coupled and tightly cohesive a design is, the better the design. Why? First, as coupling increases, the complexity of the design increases because of the possible side effects changes in one object could cause in another object. Second, a less cohesive object or method will be less well defined in what it can and should do. An object or method that does a number of things normally does none of them as well as something that has a single focus.

Poor decisions about coupling and cohesion are common in real life. For example, it would not be uncommon for a programmer to create a method for a Person object in which the Person object is read from a database. A programmer may then decide the method would be much more powerful if the user could actually manipulate the underlying data. A method could then be designed that takes a String that is the SQL select statement for the fields to be selected from a Person database table. In this case, what is returned from the method is an object array of the items the programmer has selected. This process is illustrated in the code fragment below:

```
public Object[] getPersonData(String s) ...
...
Object[] o = getPersonData("Select name, age, sex");
```

This appears to be a very powerful and useful method. Rather than forcing the programmer to write methods for each data field available in the table, this method allows the user to select just those fields in which they are interested. The problem with this type of method is that it is both too highly coupled to the internal representation of the object (the database used and the data fields in that particular database), and too loosely cohesive, as the method can be used for any query against the database. For example, you can tell that this method is too highly coupled because the query requires that the programmer using the getPersonData() method know a lot about the Person object—how the data is stored, the names of the data fields, etc. The programmer maintaining the Person object cannot change these details without affecting the programs that use this object.

This method is also too loosely cohesive in that it allows the programmer to do too many things with the object. For example, a programmer will probably realize that the object takes her select string, and appends to it to get the following query:

Select name, age, sex from Person

Given this behavior, there is no protection from an accidental (or even malicious) use of this method as follows:

Object[] o = getPersonData("Delete *");

This use of the method would delete all records in the table, which is not likely the intention of the author of the Person object. Making an object or method more loosely coupled and more tightly cohesive is almost always an advantage.

This rule does not imply that a highly cohesive or loosely coupled method or class is not complex. For example, consider the method "boolean approveMortgage(String customer, float amount)". This method would probably implement a number of interactions (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/popups/popup4-i-1.html?ou=693191) with other objects, as well as methods to determine the credit score of the person applying for the loan, to appraise the property, and so on. These interactions could even represent remote

network connects. The entire implementation of this method could be very complex. However, the program calling this method is completely unaware of the details or the complexity of the underlying transaction, so the method is loosely coupled. Also the program calling this method sees only a simple transaction of approving the mortgage, and thus, the method is highly cohesive.

Decreasing coupling and increasing cohesion in a program is often a worthy goal to aspire to when writing a program. This process can be summed up as keep things separate and simple, and the more separate and simple, the better.

II. Choosing Composition or Classification

This section will explore the decision to use composition or classification in a design. It will not discuss the design of objects, but instead deal with program errors that can occur when classification is chosen rather than composition. The final result—that you should always favor composition over classification—will be justified by showing that classification tends to create rigid and fragile designs. However, classification does have its uses, and we provide rules for when to properly use classification later in this module.

A. Problems with Classification

The easy answer to whether to use classification or composition is to simply state that, in Java, combing classes using classification (or inheritance) creates one object, a single entity that represents all properties of that object. Building an object by combining objects from multiple classes (composition) would create a design where the parts of the object are physically separate entities. Designs with single entities will generally be more tightly coupled and loosely cohesive than ones that make use of multiple entities. As a general rule, designs that use composition are safer than designs that use classification. But are designs that use composition more loosely coupled and tightly cohesive than designs that use classification? If so, which types of problems are avoided by using composition rather than classification? If these issues are not understood, programmers will not know how to use the rule, and perhaps more importantly, will not know how to break this rule effectively and safely.

As with any rule, the rule regarding composition and classification exists because it is useful. Understanding why a rule is useful, and why it was invented, is always more useful than trying to remember all the situations in which the rule applies and all the possible exceptions to the rule. Once you know why something works, you can more easily decide how to apply it. This section will show you how the coupling that occurs in classification leads to problems. This will help you avoid these problems and, just as importantly, to understand when classification represents a proper solution to a problem.

Do Not Use Classification if the Type of an Object Can Change

The first rule we will give is that you should not use classification if the type of an object can change. This can be more simply stated by saying that, since the general rule is that classification is called an *is-a* relationship, this rule can be called an is-always-a extension to the is-a rule.

Violation of the classification rule can result in extremely harmful program bugs. These bugs may be hard or impossible to fix, and may have very far-reaching and significant consequences. Worse, these bugs are hard to avoid, and often only show up after a system is in production and data for the system has been compromised, often to the point where the data is irrecoverable. By the end of this section, you will see why the classification rule is a very important rule to always consider before implementing objects that can change type, and why you should avoid violating the rule if at all possible.

To understand the classification rule, consider the following test case that we can add to our unit testing of the Bank application. In this example, the customer changes from a BlueCustomer to a SilverCustomer. This change is then made in the bank so that the Customer John Doe is now a SilverCustomer.

```
import bank.*;
import java.lang.reflect.*;
import java.util.GregorianCalendar;
import java.util.Collection;

public class TestBank {
public static void main(String[] args) {
```

```
Bank bank = null;
try {
bank = Bank.getBank();

//First create the customer, then add the account;
bank.addCustomer(new BlueCustomer(
new Name("John", "Doe"),
new GregorianCalendar(1985, 3, 13)));
Account acct = new CheckingAccount(
new Name("John", "Doe"), 2800.00F);
bank.addAccount(acct);

// Now change John Doe from Blue to Silver Customer
Customer cust = bank.getCustomer(
new Name("John", "Doe"));
cust = new SilverCustomer(cust, 1000.00f);
bank.changeCustomerType(cust);

// Print out John Doe's type in the Bank
cust = bank.getCustomer(new Name("John", "Doe"));
System.out.println("Customer = John Doe is a " +
cust.getClass() + " customer");

// Print out John Doe's type in the account
cust = bank.getAccount(
acct.getAccountNumber()).getAccountOwner();
System.out.println("Customer = John Doe is a " +
cust.getClass() + " customer");

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

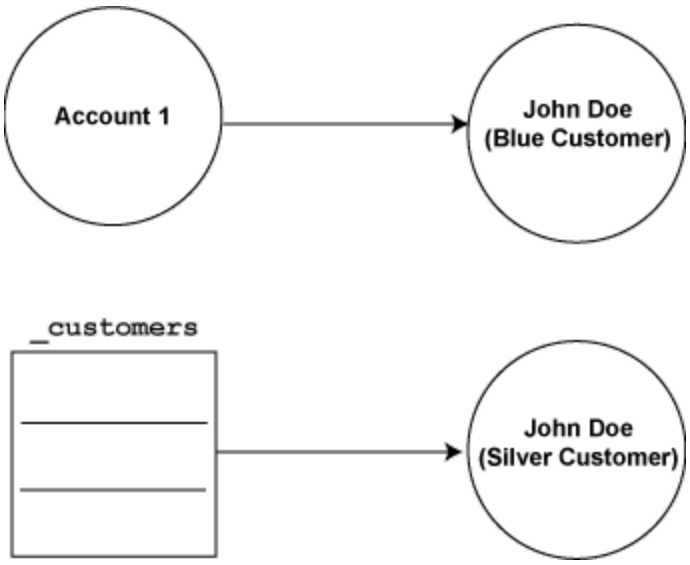
When we run this test case, we get the following result:

Customer = John Doe is a class bank.SilverCustomer customer
Customer = John Doe is a class bank.BlueCustomer customer

Now John Doe is both a BlueCustomer and a SilverCustomer in our bank. How is this possible? It is because, when the customer type for John Doe was changed, the object was forced to change types. When an object changes types, the current object is not simply changed to reflect the new data—a new object is created. This means that every reference to that object everywhere it is used in the system must be updated. The bank program did not update all references to the John Doe Customer object, and so two objects remain in the program for John Doe.

We can show this graphically in figure 4.1. In this figure, the Customer is referenced both from the _customers Map in the bank, but also from the individual accounts. When we changed the type, the reference was changed in the _customers Map, but it was not changed in the individual accounts. Thus, existing accounts will all now reference the old Customer object, and the _customers Map and any new accounts will reference the new Customer object.

Figure 4.1



References to the Customer Object in Classification Design of the Bank

This necessity of an object to change types is a problem in designs that use classification and is often not acknowledged by designers. Many designers feel that the design should represent what they, as the designer, see as the relationships in the systems. Design is often seen as being completely unrelated to the implementation in a programming language.

To many designers, the problem is that the programmer simply did not take into account all the side effects of changing a customer type. After all, no bug existed until the programmer tried to change the type, so the code that changed the type is the direct cause of the bug. Designers might feel that it is the responsibility of the programmer to account for all side effects in every change they make, regardless of the problems exposed in the design.

This expectation would be fine if it were possible to easily predict and account for all the side effects that could occur in changing a type, and if the designers took all these side effects into consideration in their designs. In a real application, however, there may be dozens of places where an object such as a Customer is referenced, and few programmers would be expected to locate each of these places. Thus, it is up to the designer to specifically catalog all of these references, and then to the project manager (not the individual programmer) to maintain this list of references, to make sure that all programmers know the places in the code that must be updated when each particular type of change is made, and to ensure that all the appropriate updates are implemented. If there are dozens of these structures to consider, and dozens of places to catalog for each structure, a reference list becomes difficult to maintain and use. This can become a major problem for the project manager and the individual programs, especially when new relationships are developed in the code, and programmers and managers are moved between projects. It is probably a fool's errant to even attempt to maintain this information, and so many times it is not maintained, and bugs crop up that a designer can conveniently attribute to the programmer, rather than to his/her own design.

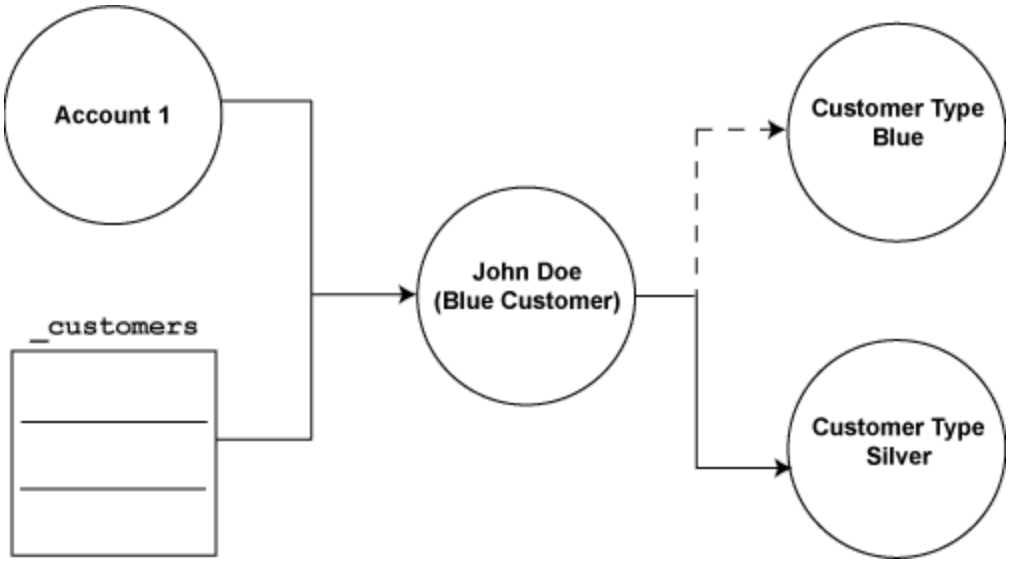
To make matters worse, the bugs created by this type of design error are also particularly insidious. For example, these bugs could occur in a production system for weeks before they are found. By then, a number of invalid objects could be

floating around the system, and trying to track down and fix the invalid data could, in itself, be difficult.

Also, the proper fix could be expensive in terms of runtime execution to fix all the bad references when changing the type. For example, in the worst case scenario for the bank, the entire _accounts collection would have to be walked in looking for a Customer who had changed types. If the bank had a large number of accounts, this would be a time-consuming and expensive operation. However, when designers use composition, they prevent all the problems that occur in changing an object's type. Using composition deters the bugs that would otherwise arise.

To see how composition prevents bugs, consider the composition design of this object, which was given in module 2 and is shown in figure 4.2:

Figure 4.2



References to the Customer Object in the Componsition Design of Bank

```
import java.util.ArrayList;
import java.util.Date;

interface CustomerType {
static float standardCharge = 25.00F;
public void processOverdraft(float amount) throws
InsufficentFundsException;
```

```
}

public class Customer {
private Name _name;
private Date _dateOfBirth;
CustomerType _custType;

ArrayList<Account> _accounts = new ArrayList<Account>();

private Customer(Name name, Date dateOfBirth, CustomerType custType) {
_name = name;
_dateOfBirth = dateOfBirth;
_custType = custType;
}

public static Customer getBlueCustomer(Name name, Date dateOfBirth) {
return new Customer(name, dateOfBirth, new BlueCustomer());
}

/* Method to retrieve name. */
public Name getName() {
return _name;
}

public void addAccount(Account account) {
_accounts.add(account);
}

public void processOverdraft(float amount) throws
InsufficentFundsException {
_custType.processOverdraft(amount);
}
}
```

In this design, the reference to the customer type still changes, but that reference is hidden inside the Customer object. The references to a Customer object in the _customers Map and the Account object are to the same Customer, and the Customer object hides the changed type. No external references ever need to change. This means that the Customer object can effectively change type without impacting the references in the rest of the program. This is extremely valuable behavior, and it is the reason for the existence of the State design pattern.

Another reason not to use classification when a type can change is that, if you need to record each time an object changes type, it is relatively easy with composition, and not so easy with classification. To understand why, again consider the Customer object. If you use composition, it is easy to set up a Collection of CustomerType objects and store each change to the CustomerType in that Collection. If, instead, you use classification, you need to store an entire Customer object, including all the data that did not change, to record any change in that Customer object.

There is a basic principle illustrated here. If many references can point to the same object, and it is expected that all the references consistently point to the same object, that reference should not be able to change. In the case discussed above, a proxy object must be created that (1) never changes its reference and (2) contains the one reference to the actual object of interest. Then, only one reference to an object needs to change to update all the referring objects.

Another problem with using classification when a history is needed is that, since a classification design results in a single instance of an object, it is impossible to separate the data for that object to just store the part that is needed for the history. When a history of only one part of an object is to be stored, classification designs are just too tightly coupled for a single object instance to be created.

Because of the reference problem and potential history problem in designs that use classification, and the fact that this problem does not occur in designs that use composition, there is seldom, if ever, a reason to use classification if your design will allow an object to change type. Unless there is some overriding criteria, it is best not to use classification when an object can change type.

Do Not Use Classification Unless You Really Want to Be an Instance of the Class You Are Extending

The second rule is not to use classification unless you really want to create an instance of the class you are extending. As was pointed out earlier, Java has true run time data typing. This means that the information about ancestor classes (for example, the parent class) is always available. Thus a programmer can write programs that rely on a class having a specific parent. However, in many cases there is not a good reason why the program should know what the class is that is extended. This is particularly true when a class is being used as a utility, to provide a service to another class.

A good example of a situation where a class is extended simply to provide a service is called representation inheritance (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/popups/popup4-ii-1.html?ou=693191) (or is-represented-by). Representation inheritance is shown in the following example. In this example, the instanceof operation shows that a Stack is a Vector, since it has extended Vector to represent the collection to store the elements in the Stack.

```
import java.util.*;

public class ShowStack {
public static void main(String args[]) {
Stack s = new Stack();
if (s instanceof Vector)
System.out.println("True");
}
}
```

This program raises the question, why does a Stack need to know whether or not it is a Vector? It is far more likely that this type of construction will be misused than that it will be used for a valid purpose. In fact, the larger question is, why should the type of data structure used to store the data in the Stack be available to the program at all?

In fact, the data structure used by a stack should probably not be available outside of the Stack class. That it is known to the implementing program that a Stack is a Vector is generally considered to be a bug in the Stack class as implemented in Java. One of the concrete results of the knowledge that a Vector is used by the Stack class is that Stack cannot be upgraded to use the new JDK 1.2 Collection classes, such as ArrayLists. However, other components that store collections of data, such as a Button, can be updated to use the new collections. The reason why Button can be upgraded and Stack cannot is that Button does not advertise what type of mechanism it uses to store its Collection data, so the Collection can be changed; a Stack advertises its Collection type (a Vector) through inheritance. Thus a Stack in Java is a Vector, and to change that fact changes what it is, which might cause some programs to fail.

For another example of this problem, consider the following simple GUI program. In it, the GUI has mixed Swing and Another Windowing Toolkit (AWT) components. AWT was the original GUI toolkit that came with Java starting with the J2SDK 1.0. Swing was an improved GUI toolkit that became available with J2SDK 1.3. The two GUI toolkits use mutually incompatible methods to access the underlying Operating System, and as anyone who has ever accidentally mixed these two types of components knows, the result is components that do not behave correctly in an application.

So why, if the toolkits are mutually incompatible, are the components from one toolkit allowed to interact with components from the other toolkit? This interaction occurs because components from both GUI toolkits extend java.awt.Component. The toolkits have some components in common; for example, the layout managers, which use the Component directly rather than separating the interface (which could have been consistent) from the type of component, which really does not have to be Component.

```
import java.awt.*;
import javax.swing.*;

public class ShowButton {
public static void main(String[] args) {
```

```
JFrame frame = new JFrame();
Container con = frame.getContentPane();

Button b = new Button("Wrong type");
con.add(b);

frame.pack();
frame.setVisible(true);
}
```

The basic principle displayed here is that, in Java, an object type is important. The contents of object say something about that object because Java keeps that information via runtime typing. Therefore when the extends clause (e.g. implement classification) is used in Java, something is said about the data type of the object, and so extends should be avoided unless that information is truly intended to be conveyed outside of the object.

Do Not Inherit Trouble Just Because It Is Free

One of the big arguments for the use of classification is that all the capability of the base class can be used by any subclass. In effect, this behavior is free to the subclass. Many programmers believe that, if something is free, you take it. But if what you are getting for free is trouble, it is a poor bargain.

For example, consider the follow example of using a Stack. Because Stack inherits from Vector, you get many methods for free, including an add() method. Although a programmer should know that the add() method should never be used for a stack (the only valid methods to manipulate the contents of a stack should be pop(), push(), and peek()), there are times in writing a program when a programmer simply types the wrong thing. A mistake of this type is shown in the program below, where a programmer[?] inadvertently uses the add() method instead of the push() method. This results in the stack storing the data in the wrong order.

```
import java.util.*;

public class ShowStack {
public static void main(String args[]) {
```

```
Stack<Integer> s = new Stack<Integer>();
s.push(1);
s.add(0,2);
System.out.println(s.pop());
System.out.println(s.pop());
}
}
```

Again, some programmers argue that this is not a problem with inheritance, but that the person using the Stack should be more careful. But why should it be easy to make an error that can easily be avoided? One of the main purposes of modern compilers and runtime systems is to recognize that programmers are human, and to protect them from silly and careless mistakes. This is especially true when all that the free behavior gets for you is the ability to call methods, such as the add() method above, which are seldom, if ever, used in a correct program. And, if such behavior is really needed, it can be accessed safely through Delegation anyway. Just because there is behavior in the Stack class that you can inherit for free, that does not mean that you should take it without considering the risks.

Another example of inheriting trouble can be noted when examining the Java GUI's, particularly Swing. Until recent versions of Swing, the following program would have resulted in a runtime error. The reason for this is that you cannot add components directly to a JFrame in Swing; you need to add them to one of the containers, such as the ContentPane, that are in the JFrame.

If you cannot add components directly to the JFrame, why then does the JFrame have an add() method? The add() method is present because it is inherited from Frame. But it serves no real purpose in the JFrame.

Recent versions of Java make any calls to the JFrame's add() method reference the ContentPane by default, but this does not change the fact that this is simply a kludge that was added to the system when it was determined that the add() method in JFrame was a mistake.

```
import java.awt.*;
import javax.swing.*;

public class ShowButton {
public static void main(String[] args) {
JFrame frame = new JFrame();
Container con = frame.getContentPane();

JButton b = new JButton("Something");
frame.add(b);

frame.pack();
frame.setVisible(true);
}
}
```

Another problem with inheriting unneeded functionality is that it results in bloated designs. For example, a JButton has over 200 methods, many of which you cannot and should not ever call.

This leads to the rule that you should not use classification simply to inherit the behavior of the parent—or, as stated above, do not inherit trouble simply because it is free. You should instead use composition and then use delegation to access the behavior you need.

Consider How the Class Will Be Stored and Used

Because classification couples classes to create one object, that object must represent a single entity. As a single entity, the object must reside in a single address space on a single computer. This can have a number of implications in, for example, a distributed application.

Consider the Bank application that used classification to design the Customer class. In the Bank application, an assumption was made that the code to check a Customer object for an overdraft and penalty would exist on the same computer as the code to store the base data for the Customer. Where was this assumption made? Because there is one object and that object has one reference, and a single

reference can only exist on a single computer, there is an implicit assumption that all checking for overdrafts will be done on the same computer that has stored the Customer object.

The compositional design, however, makes no such assumption about where the part of the Customer object that checks for overdraft and penalty exists. The CustomerType interface simply says that there will be a method that will be accessible on an object. Whether that object is on a local computer, or represents some remote service that exists on a totally separate computer, is not part of the description. The object that supplies the services for the CustomerType interface is completely decoupled from the Customer object, and thus can be distributed anywhere.

Even if, after the initial design is created, it is known that the CustomerType will be accessed on the local computer, it is still best to design the class using composition. This allows the design to adapt to changes in how the CustomerType is accessed. In general, unless there is something to be gained from using classification, it is always best to leave open options such as having the design become distributed.

Carefully Consider All Calls to Non-Final Virtual Methods

As was shown in module 1, virtual methods are methods that are always called on the actual type of the object. Since all methods in Java are virtual, if a non-final method is called, there is no guarantee that the method with that name in the current class will be called. If that method is overridden in any subclass, the overridden method will be called instead.

Virtual methods are often used by programmers to allow for the use of additional behavior in a subclass that is not provided in the base class. However, virtual methods should be used carefully because they allow for the possibility of abuse. If there is some set of conditions in the base class method on which the object relies, it is very possible for the child class to write the overridden method and remove that behavior. Once again, this cannot happen in a compositional design because the contained objects are delegated work, which is controlled by the encapsulating object.

Overriding virtual methods is not, in itself, a problem. The toString() method has been overridden frequently in these modules, and with good result. The place of virtual methods in a program, however, is something a programmer should be aware of and should use carefully.

Where this behavior does present a problem is in methods called by a constructor. The call to the constructor is supposed to be the first step in creating an instance of a class. But when a constructor calls a virtual method, and that virtual method is overridden in the subclass, the method of that subclass is executed before the object for that subclass has been constructed. This can lead to some strange behavior.

To view this behavior, consider the following simple classes. See that class B extends class A, and class A calls a method named method1() in its constructor. The method method1() simply sets the variable _var to 1. Subsequently, class B overrides method1(), but does not set the variable _var until it completes its constructor, when it sets _var to 2. So, what value of _var is printed out in the constructor for A? Zero (0). This is because all object variables are initialized to zero when the object is created, and no method has been called that changes this value before the constructor in A prints it out. This particular problem is very confusing, even to experienced programmers, because a properly constructed instance of A will have the value of _var set to 1, and a properly constructed instance of B will have a value of _var set to 2. During its construction of the object B, however, the base class constructor for A will have a transitory value of _var, which is invalid. If the constructor is relying on that value to be valid, the program will not work correctly.

```
public class ShowVirtual {
public static void main(String[] args) {
B b = new B();
}
}

class A {
int _var;
```

```
public A() {
System.out.println(_var);
}

public void method1() {
_var = 1;
}
}

class B extends A {
int _var;
public B() {
_var = 2;
}

public void method1() {
}
}
```

This type of problem arises frequently with virtual methods called from a constructor. It is impossible to ensure that the object is initialized before it is used, and in Java, that can lead to many strange results.

It is important to understand that this type of problem also occurs in concurrent programs. Concurrent programs are programs that use more than one thread of execution, and will be covered in more detail in module 5. However, concurrent programs are more prevalent than many programmers would like to believe. Any program that uses a GUI in Java is concurrent, and care must be taken when writing for concurrency to avoid program bugs.

In a concurrent program, if a thread is somehow created in a constructor, that thread has access to the data of the object even though that object might not yet have been constructed. This can lead to situations even stranger than the virtual method problem described above.

While the arguments here are not complete, we hope that they are sufficient to make the programmer carefully consider how classification through virtual methods can create situations where bugs can occur, and to carefully consider how to best implement and use virtual methods.

Know What You Are Sharing When You Share an Object Reference

Most Java programmers are aware that all objects in Java extend class Object. Most programmers, however, are unaware as to why objects in Java have this property, and the implications of the fact that all objects in Java are instances of class Object. For example, the class Object contains hidden data fields, such as a monitor, tha we will discuss in module 5.

When using classification to extend a class, it is once again important for the programmer to realize that he/she has created one, and only one, instance of the class Object. Thus, the base- and subclasses together share one common set of data for an Object. As we will show in module 5, the monitor field in the object is shared. This sharing greatly affects synchronized statements, which are used to safely ensure access to parts of an object.

Suppose, for example, a synchronized statement exists in a base class, and that base class is extended. If the subclass also chooses to use the synchronized statement, there will be only one monitor from the shared instance of Object that both classes have inherited. This means that the two classes will interfere with each others' use of the monitor. This interference can result in bugs that are extremely difficult to diagnose and fix, as will be seen in module 5.

In a composition design, these bugs simply do not occur because, when you create an object from a composite class, each object is separate. There is one object for each class. Therefore, each object is its own instance of class Object, and does not share the properties from class Object. Because no interactions occur between the classes, it is generally safer to simply ignore how the base class Object actually behaves. You do not need to consider how sharing this hidden data affects your application.

Once again, you should not couple classes into a single object unless you know what features you are coupling. When you extend any class in Java, you are also extending one instance of class Object. Composition is always safer then, because the objects are separate and do not share the data from class Object.

B. Choosing Composition or Classification

In the discussion so far, we have constantly stressed that composition should, in most circumstances, be favored over classification. The designs produced by composition tend to be less rigid and more stable and extensible. So, how do you choose which to use, classification or composition? In general, start your designs with composition, and use classification when you can show a discernable benefit.

This does not mean that classification should never be used. In fact, there are a number of situations in which classification designs are not simply better, but the application could not be effectively implemented without them. These designs tend to have the following four characteristics in common:

1. The objects do not change types. For example, immutable objects cannot change value, and therefore cannot change type.

2. The type is a true subtype. By this, we mean that the subtype is everything the base type is, with some extra functionality. Typically, this subtype relationship acts in a manner similar to a transaction.

3. There is functionality in the base type that can apply across all subtypes.

4. The type of the object, both the base type and subtype, needs to be exposed.

Two excellent examples of types that fit these criteria are java.lang.Exception and java.util.Event. In both cases, the objects created are immutable, so there is no concern about the type ever changing. In both cases, the types created are true subclass relationships; they are the base class plus extra information and functionality. In both cases, there is processing provided by the base type that is used in the subtypes (e.g., printStackTrace in the Exception class). Finally, in both cases, the program needs to know what the entire type is, both the base and parent

type. The base type is needed for generic handling (in particular, in the case of the Exception, to determine if the Exception is checked or unchecked), and the subtype is needed by the application which implements the Exception or Event.

There are a number of other examples in Java, so a programmer should always be open to the possibility of using classification. But in the end, unless there is a tangible benefit to using classification, composition should always be favored over classification.

III. Aggregation and Acquaintance

Designs that use composition are, generally speaking, safer than designs that use classification. However, this does not mean that there are not problems inherent to composition. In particular, the aptitude with which an object isolates information is critical to the design of a safe system. This section will deal with how to isolate data when using composition to create safer designs.

A. Encapsulation

Encapsulation (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/popups/popup4-iii-1.html?ou=693191) in OOP is the inclusion of one object within another thing so that the included object is not visible outside of the encapsulating object. For example, the following definition of a Name class is not encapsulated because the instance variables _firstName and _lastName are both visible and can be accessed from outside of the Name class.

```
public class Name {
public String _firstName;
public String _lastName;

public void setName(string firstName, string lastName) {
_firstName = firstName;
_lastName = lastName;
}
```

```
public String toString() {
return _firstName + " " + _lastName;
}
}
```

To encapsulate the instance variables, it is necessary to make them private so that they are hidden in the object, and to provide access to those hidden instance variables through methods. The following source code shows a properly encapsulated Name class.

```
public class Name {
private String _firstName;
private String _lastName;

public void setName(string firstName, string lastName) {
_firstName = firstName;
_lastName = lastName;
}

public String toString() {
return _firstName + " " + _lastName;
}
}
```

Encapsulation is the core mechanism responsible for ensuring managed access to an object and can be used in a number of ways. For example, it can be used to limit the range of values for a variable, as in the Person class below, where the age is limited to values between 0 and 125. This is an important addition to program safety.

```
public class Person {
private String _age;

private final boolean checkAge(int age) {
if (age > 0 && age < 125)
return true;
```

```
else
return false;
}

public Person(int age) {
if (checkAge(age)
_age = age;

else
_age = 0;
}

public void setAge(int age) {
if (checkAge(age)
_age = age;

else
_age = 0;
}
}
```

The ability to manage access to a variable has even stronger implications in program safety, as will be seen in the following sections.

If encapsulation is such a good thing, why doesn't a language such as Java make it required? There could be many reasons. One is that making a program work (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/popups/popup4-iii-2.html?ou=693191) sometimes requires that direct access to a variable be provided. The situations in which direct access is needed are often obscure, rare, and hard to explain. For example, some systems rely on a concept called *reflection*, which requires that the JVM actually have access to instance variables in the program. An example of reflection is used in Java Jini, which is an advanced distributed programming environment.

When the JVM needs to directly access instance variables, Java security requires that all variables in class be public variables. The reasons for this are beyond the scope of this class, but the idea is clear. There are cases, such as when reflection is being used, where a programmer cannot use encapsulation on instance variables. Generally speaking, however, a class should always implement encapsulation on its instance variables unless there is some overriding reason for not using it.

B. Aggregation and Acquaintance

Encapsulation is the first step toward isolating the instance variables inside a class. This principle will now be used to implement aggregation and acquaintance (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/popups/popup4-iii-3.html?ou=693191) . Gamma et al. (1995, 22–23) define aggregation and acquaintance as follows:

"Aggregation implies that one object owns or is responsible for another object. Generally we speak of an object having or *being part of* another object. Aggregation implies that an aggregate object and its owner have identical lifetimes."

"Acquaintance implies that an object merely *know of* another object. Sometimes acquaintance is called "association" or the "using" relationship. Acquainted objects may request operation of each other, but they aren't responsible for each other."

In the rest of this section, we will look at these distinctions between aggregation and acquaintance.

C. Acquaintance

Acquaintance represents a weaker control of one object over another, so we will explore it first. In an acquaintance relationship, one object knows about another object, so the encapsulating object stores a reference to the acquaintance object. However the acquaintance object has an existence (or has references) that can be modified outside of the encapsulating object. To understand this, consider the following example of a Car that encapsulates a Driver object. It is apparent from this relatively simple program that, while the Car object properly encapsulates

the Driver object, the Driver object can still be changed without the explicit knowledge of the encapsulating Car object. The ability to access and modify an object outside of the encapsulating object is an important design consideration because it implies that the Driver of the Car can be changed (or at least the Driver's name) without the Car object being aware of that change.

```
class Name {
private String _firstName;
private String _lastName;

public Name(String firstName, String lastName) {
_firstName = firstName;
_lastName = lastName;
}

public void setName(String firstName, String lastName) {
_firstName = firstName;
_lastName = lastName;
}
}

class Car {
private Name _driver;

public Car(Name driver) {
_driver = driver;
}
}

public class CarAndDriver {
public static void main(String[] args) {
Name name = new Name("Mario", "Andretti");
{
Car car = new Car(name);
name.setName("Dale", "Earnhardt");
```

```
} // <-- Note at this point car is out of scope, and
// can be garbage collected, but Name cannot.
}
}
```

The ability to modify an encapsulated object is important in many designs. There are designs that cannot exist without this property. But it is important for a programmer to recognize when an acquaintance relationship is being used, as the encapsulating object cannot be responsible for the state of the acquaintance object in a design. As we will show later, this relationship has even wider consequences, as programmers often do not even recognize that an acquaintance relationship has been implemented.

D. Aggregation

What is Aggregation?

In aggregation, the encapsulating object owns the aggregate object. What is meant by this is that the aggregate object cannot be changed outside of the encapsulating object, and thus, the encapsulating object can make guarantees about the state of the aggregate object. For example, consider the following implementation of a Car, where the _engine object is aggregated with the object. The _engine object is created in the Car constructor. Since no reference to the _engine object exists outside of this Carobject, all changes to the _engine object must be done through the Car. This is an important design consideration. It means that, unlike the previous Car class's Driver object, this Car class can make guarantees about the state of the _engine object.

```
class Engine {
private int _size;

public Engine(int size) {
_size = size;
}
}

class Car {
```

```
private Engine _engine;

public Car(int engineSize) {
_engine = new Engine(engineSize);
}
}

public class CarAndEngine {
public static void main(String[] args) {
Car car = new Car(350);
} //<- Note that car goes out of scope, and so does _engine,
// So both can be garbage collected.
}
```

Note that there are other possible ways to create an _engine object for a Car and still use aggregation. One is to pass an Engine object into the Car constructor, or any other method, and simply then clone the Engine object. Since this means that the Engine object stored in the Car class is a different object from the one passed in as a parameter, this Engine object now has no reference outside of the encapsulating object, resulting in an aggregation relationship. Note that this same technique must be followed if an aggregate object is ever returned from a method in the encapsulating object. If a clone is not made, the reference has escaped the encapsulating object, and the programmer has created a possibly incorrect acquaintance relationship.

```
class Engine implements Cloneable {
private int _size;

public Engine(int size) {
_size = size;
}

public Engine clone() {
return new Engine(this._size);
}
```

```
}

class Car {
private Engine _engine;

public Car(Engine engine) {
_engine = engine.clone();
}

public Engine getEngine() {
return _engine.clone();
}
}

public class CarAndEngine {
public static void main(String[] args) {
Engine engine = new Engine(350);
{
Car car = new Car(engine);
} //<- Note that car goes out of scope, and so
// does _engine, so both can be garbage
// collected. However the engine created
// in this main method is still in scope
// and cannot yet be garbage collected.
}
}
```

One final note on when an aggregate object can be garbage collected: in these programs, because there is no reference to the aggregate object outside of the encapsulating object, when the encapsulating Car object is garbage collected, the only reference to the aggregate _engine object should be removed so that the aggregate object can also be garbage collected. It would seem logical to interpret Gamma's words, "Aggregation implies that an aggregate object and its owner have

identical lifetimes," to refer to the fact the aggregate _engine object was created at the same time as the encapsulating Car object. Whether this is what Gamma truly meant will be the subject of the next section.

Aggregation and Lifetimes of Objects

The terminology used in Gamma's definition, that an "aggregate object and its owner have identical lifetimes," is used to make a point about aggregates, but it is misleading. The point is that an aggregate object is completely owned by the encapsulating object, and so when the encapsulating object goes out of scope, the aggregate object (typically) also goes out of scope, because it is owned by (or has no reference outside of) the encapsulating object. Thus, when an encapsulating object is garbage collected, any aggregated objects are (generally) garbage collected. An acquaintance object, on the other hand, cannot be garbage collected when the encapsulating object is garbage collected because, by definition, an acquaintance object has a life outside of the encapsulating object.

Gamma's definition implies that no reference to an aggregated object exists outside of the encapsulating object. This is true, but the statement that an "aggregate object and its owner have identical lifetimes" is too limiting. For example, consider the following simple Car class, where an Engine can be replaced in the Car. In this Car class, neither the old Engine nor the new Engine objects have a reference outside of the Car, and thus, they are, appropriately, aggregation relationships with the Car object. This is properly aggregation because the Car object can make guarantees about the behavior of both Engine objects, yet neither of the Engines has the same lifetime as the Car object.

```
class Engine implements Cloneable {
private int _size;

public Engine(int size) {
_size = size;
}

public Engine clone() {
return new Engine(this._size);
```

```
}
}

class Car {
private Engine _engine;

public Car(Engine engine) {
_engine = engine.clone();
}

public Engine getEngine() {
return _engine.clone();
}

public void replaceEngine(Engine engine) {
_engine = engine.clone();
}
}
```

An even weaker aggregation relationship exists where the object to be aggregated is immutable. For example, consider the following definition of a Car class where, in the Car constructor, a String parameter for the modelType variable is provided. Obviously, the String object has a reference outside of the Car; however, because String is immutable, there is no problem in simply setting the modelType instance variable to the same reference as the parameter. To have to say "_modelType = new String(modelType);" is simply overkill in this instance.

```
class Car {
private String _modelType;

public Car(String modelType) {
_modelType = modelType;
}
}
```

E. When to use Aggregation and Acquaintance

This all leads to the question of when to use aggregation and acquaintance, and when to make the aggregate object immutable. To determine the answer, we must first consider that a type that implements an immutable object is an aggregation relationship that has additional constraints; specifically that the data values cannot be changed. Thus, immutable objects are the safest objects to use in a program. Second, we must realize that, in acquaintance relationships, references are available outside of the encapsulating object; therefore, aggregation relationships are safer than acquaintance relationships.

So, what criteria can be used to choose which relationship to implement? A good rule of thumb is to start with the most stringent (safest) design, and to loosen these constraints when the need arises. The safest course is to first assume that all classes in a system should be immutable. Obviously, this cannot be true, but classes should be changed from ones that are immutable to ones that are aggregate only when the need arises. As the design continues to evolve, it will become apparent that certain classes need to be acquaintance; however, only when the need arises should a class that uses an aggregate relationship be made into one that uses an acquaintance relationship.

F. Recognizing Acquaintance Relationships

One major problem in implementing classes is that it is often easy for a relationship intended to be an aggregation relationship to degenerate accidentally into one that is an acquaintance relationship. Consider the Customer class used in the case study in module 3. In the Customer class, there is an instance variable _accounts that is a collection of Account objects owned by this Customer object. When the GUI was implemented, there was a need to display these Account objects, so a getAccounts() method was innocently added that simply returned this Collection. The problem with this addition is that now, the _accounts Collection is no longer an aggregate of the Customer object. Any programmer (either through ignorance, haste, or malicious intent) can access this Collection and modify it. So, although adding the getAccounts() method did not result in any immediately apparent program errors, the addition must be considered a bug in the program.

How can this problem be avoided? Earlier, we saw that when you want to pass data out of a method without violating an aggregation relationship, you can simply clone the object(s) that you wish to return. This can be done as follows:

```
public Collection<Account> getAccounts() {
return (Collection<Account>)_accounts.clone();
}
```

This method works well for relatively small Collections that are not frequently accessed, but if the number of items grows in this Collection, cloning it each time it must be returned can become prohibitively expensive. Java has provided a way to wrapper a Collection so that it cannot be modified. You can create this wrapper by calling the Collections.unmodifiableCollection() method, as illustrated in the code fragment below.

```
public Collection<Account> getAccounts() {
return (Collection<Account>)
(Collections.unmodifiableCollection(_accounts));
}
```

Note that this Collection technically violates the rule that, for an aggregate object, a reference to the object should not exist outside of the encapsulating object. However, the procedure of using a wrapper can provide a significant performance increase over cloning, and it does keep the aggregate object safe. The wrapper achieves the necessary safety constraints of an aggregate object at a much lower cost than by cloning.

The above example illustrates why it is important to understand why rules are created, and to not simply memorize the rules and mechanically apply them. The unmodified collection does, indeed, violate the rule about aggregate data having the same lifetime as the encapsulating object. But blindly following this rule could result in a program that is inefficient and not useful.

If one idea is taken away from this module (and indeed this class), it should be that, in all design decisions, creating a safe and effective design is more important than following a set of rules. In the final analysis, a programmer has to do what he/she

must to field a program. If a programmer understands why certain rules exist, he or she can apply the principles behind the rules to make sound decisions that will enhance and safeguard a program.